plement the construct by the equivalent "not exists X such that not $P(X)$" construct, which can be written in SQL. We write

```
create assertion sum-constraint check
    (not exists (select * from branch
        where (select sum(amount) from loan
                where loan.branch-name = branch.branch-name)
            >= (select sum(balance) from account
                where account.branch-name = branch.branch-name)))

create assertion balance-constraint check
    (not exists (select * from loan
        where not exists ( select *
            from borrower, depositor, account
            where loan.loan-number = borrower.loan-number
                and borrower.customer-name = depositor.customer-name
                and depositor.account-number = account.account-number
                and account.balance >= 1000)))
```

When an assertion is created, the system tests it for validity. If the assertion is valid, then any future modification to the database is allowed only if it does not cause that assertion to be violated. This testing may introduce a significant amount of overhead if complex assertions have been made. Hence, assertions should be used with great care. The high overhead of testing and maintaining assertions has led some system developers to omit support for general assertions, or to provide specialized forms of assertions that are easier to test.

6.4 Triggers

A **trigger** is a statement that the system executes automatically as a side effect of a modification to the database. To design a trigger mechanism, we must meet two requirements:

1. Specify when a trigger is to be executed. This is broken up into an *event* that causes the trigger to be checked and a *condition* that must be satisfied for trigger execution to proceed.
2. Specify the *actions* to be taken when the trigger executes.

The above model of triggers is referred to as the **event-condition-action model** for triggers.

The database stores triggers just as if they were regular data, so that they are persistent and are accessible to all database operations. Once we enter a trigger into the database, the database system takes on the responsibility of executing it whenever the specified event occurs and the corresponding condition is satisfied.

6.4.1 Need for Triggers

Triggers are useful mechanisms for alerting humans or for starting certain tasks automatically when certain conditions are met. As an illustration, suppose that, instead of allowing negative account balances, the bank deals with overdrafts by setting the account balance to zero, and creating a loan in the amount of the overdraft. The bank gives this loan a loan number identical to the account number of the overdrawn account. For this example, the condition for executing the trigger is an update to the *account* relation that results in a negative *balance* value. Suppose that Jones' withdrawal of some money from an account made the account balance negative. Let t denote the account tuple with a negative *balance* value. The actions to be taken are:

- Insert a new tuple s in the *loan* relation with

  $$s[\textit{loan-number}] = t[\textit{account-number}]$$
  $$s[\textit{branch-name}] = t[\textit{branch-name}]$$
  $$s[\textit{amount}] = -t[\textit{balance}]$$

  (Note that, since t[*balance*] is negative, we negate t[*balance*] to get the loan amount—a positive number.)

- Insert a new tuple u in the *borrower* relation with

  $$u[\textit{customer-name}] = \text{“Jones”}$$
  $$u[\textit{loan-number}] = t[\textit{account-number}]$$

- Set t[*balance*] to 0.

As another example of the use of triggers, suppose a warehouse wishes to maintain a minimum inventory of each item; when the inventory level of an item falls below the minimum level, an order should be placed automatically. This is how the business rule can be implemented by triggers: On an update of the inventory level of an item, the trigger should compare the level with the minimum inventory level for the item, and if the level is at or below the minimum, a new order is added to an *orders* relation.

Note that trigger systems cannot usually perform updates outside the database, and hence in the inventory replenishment example, we cannot use a trigger to directly place an order in the external world. Instead, we add an order to the *orders* relation as in the inventory example. We must create a separate permanently running system process that periodically scans the *orders* relation and places orders. This system process would also note which tuples in the *orders* relation have been processed and when each order was placed. The process would also track deliveries of orders, and alert managers in case of exceptional conditions such as delays in deliveries.

6.4.2 Triggers in SQL

SQL-based database systems use triggers widely, although before SQL:1999 they were not part of the SQL standard. Unfortunately, each database system implemented its

```
create trigger overdraft-trigger after update on account
referencing new row as nrow
for each row
when nrow.balance < 0
begin atomic
   insert into borrower
          (select customer-name, account-number
           from depositor
           where nrow.account-number = depositor.account-number);
    insert into loan values
          (nrow.account-number, nrow.branch-name, − nrow.balance);
   update account set balance = 0
          where account.account-number = nrow.account-number
end
```

Figure 6.3 Example of SQL:1999 syntax for triggers.

own syntax for triggers, leading to incompatibilities. We outline in Figure 6.3 the SQL:1999 syntax for triggers (which is similar to the syntax in the IBM DB2 and Oracle database systems).

This trigger definition specifies that the trigger is initiated *after* any update of the relation *account* is executed. An SQL update statement could update multiple tuples of the relation, and the **for each row** clause in the trigger code would then explicitly iterate over each updated row. The **referencing new row as** clause creates a variable *nrow* (called a **transition variable**), which stores the value of an updated row after the update.

The **when** statement specifies a condition, namely $nrow.balance < 0$. The system executes the rest of the trigger body only for tuples that satisfy the condition. The **begin atomic** ... **end** clause serves to collect multiple SQL statements into a single compound statement. The two **insert** statements with the **begin** ... **end** structure carry out the specific tasks of creating new tuples in the *borrower* and *loan* relations to represent the new loan. The **update** statement serves to set the account balance back to 0 from its earlier negative value.

The triggering event and actions can take many forms:

- The triggering *event* can be **insert** or **delete**, instead of **update**.

  For example, the action on **delete** of an account could be to check if the holders of the account have any remaining accounts, and if they do not, to delete them from the *depositor* relation. You can define this trigger as an exercise (Exercise 6.7).

  As another example, if a new *depositor* is inserted, the triggered action could be to send a welcome letter to the depositor. Obviously a trigger cannot directly cause such an action outside the database, but could instead add a tuple to a relation storing addresses to which welcome letters need to be sent. A separate process would go over this table, and print out letters to be sent.

- For updates, the trigger can specify columns whose update causes the trigger to execute. For instance if the first line of the overdraft trigger were replaced by

  create trigger *overdraft-trigger* **after update of** *balance* **on** *account*

  then the trigger would be executed only on updates to *balance*; updates to other attributes would not cause it to be executed.

- The **referencing old row as** clause can be used to create a variable storing the old value of an updated or deleted row. The **referencing new row as** clause can be used with inserts in addition to updates.

- Triggers can be activated **before** the event (insert/delete/update) instead of **after** the event.

  Such triggers can serve as extra constraints that can prevent invalid updates. For instance, if we wish not to permit overdrafts, we can create a **before** trigger that rolls back the transaction if the new balance is negative.

  As another example, suppose the value in a phone number field of an inserted tuple is blank, which indicates absence of a phone number. We can define a trigger that replaces the value by the **null** value. The **set** statement can be used to carry out such modifications.

  create trigger *setnull-trigger* **before update on** *r*
  referencing new row as *nrow*
  for each row
  when *nrow.phone-number* = ' '
  set *nrow.phone-number* = **null**

- Instead of carrying out an action for each affected row, we can carry out a single action for the entire SQL statement that caused the insert/delete/update. To do so, we use the **for each statement** clause instead of the **for each row** clause.

  The clauses **referencing old table as** or **referencing new table as** can then be used to refer to temporary tables (called *transition tables*) containing all the affected rows. Transition tables cannot be used with **before** triggers, but can be used with **after** triggers, regardless of whether they are statement triggers or row triggers.

  A single SQL statement can then be used to carry out multiple actions on the basis of the transition tables.

Returning to our warehouse inventory example, suppose we have the following relations:

- *inventory(item, level)*, which notes the current amount (number/weight/volume) of the item in the warehouse

```
create trigger reorder-trigger after update of amount on inventory
referencing old row as orow, new row as nrow
for each row
when nrow.level <= (select level
                    from minlevel
                    where minlevel.item = orow.item)
and orow.level > (select level
                  from minlevel
                  where minlevel.item = orow.item)
begin
    insert into orders
        (select item, amount
         from reorder
         where reorder.item = orow.item)
end
```

Figure 6.4 Example of trigger for reordering an item.

- *minlevel(item, level)*, which notes the minimum amount of the item to be maintained
- *reorder(item, amount)*, which notes the amount of the item to be ordered when its level falls below the minimum
- *orders(item, amount)*, which notes the amount of the item to be ordered.

We can then use the trigger shown in Figure 6.4 for reordering the item.

Note that we have been careful to place an order only when the amount falls from above the minimum level to below the minimum level. If we only check that the new value after an update is below the minimum level, we may place an order erroneously when the item has already been reordered.

Many database systems provide nonstandard trigger implementations, or implement only some of the trigger features. For instance, many database systems do not implement the **before** clause, and the keyword **on** is used instead of **after**. They may not implement the **referencing** clause. Instead, they may specify transition tables by using the keywords **inserted** or **deleted**. Figure 6.5 illustrates how the overdraft trigger would be written in MS-SQLServer. Read the user manual for the database system you use for more information about the trigger features it supports.

6.4.3 When Not to Use Triggers

There are many good uses for triggers, such as those we have just seen in Section 6.4.2, but some uses are best handled by alternative techniques. For example, in the past, system designers used triggers to maintain summary data. For instance, they used triggers on insert/delete/update of a *employee* relation containing *salary* and *dept* attributes to maintain the total salary of each department. However, many database systems today support materialized views (see Section 3.5.1), which provide a much